Part 1: Service Architecture & Setup

1) *Explain the default lifecycle of a JAX-RS Resource class. Is a new instance instantiated for every incoming request, or does the runtime treat it as a singleton? Elaborate on how this architectural decision affects how you manage and synchronize your in-memory data structures (maps/lists) to prevent data loss or race conditions.*

    - JAX-RS creates resource classes on demand, resulting in a new object being instantiated with each request instead of having one common singleton instance.
    - This implies that instance variables cannot be shared among different requests, making it impossible to store any data within resource objects. Therefore, in this project, we have made use of the DataStore object containing static variables of type HashMap/ArrayList.
    - Nevertheless, sharing static variables may result in concurrency issues, which need to be solved in a real environment using thread-safe data structures such as ConcurrentHashMap.

2) *Why is the provision of" Hypermedia" (links and navigation within responses) considered a hallmark of advanced RESTful design (HATEOAS)? How does this approach benefit client developers compared to static documentation?*

    - The use of hypermedia (HATEOAS) is seen as one of the characteristics of a well-designed REST architecture because it enables the response of an API to contain links to other resources. The client will be able to discover the available actions and links without having to make any assumptions about the endpoint. The API becomes self-documenting and very flexible for both the client and developer. Developers are able to develop the client without the need for external documentation. Any changes made in the endpoints will not affect the client because of the discovery process.

Part 2: Room Management

1) *When returning a list of rooms, what are the implications of returning only IDs versus returning the full room objects? Consider network bandwidth and client-side processing.*

    - The benefit of returning room IDs only is that it leads to smaller response sizes and thus uses less network bandwidth and makes the operation more efficient, especially considering that we need to work with very large datasets. Nevertheless, it means that there will be an additional call from the client to obtain all room details, adding overhead to the process. Returning whole room objects provides the required data in one request, which facilitates its use by the client, although it also means larger payloads and possibly lower performance when handling large amounts of data.

2) *Is the DELETE operation idempotent in your implementation? Provide a detailed justification by describing what happens if a client mistakenly sends the same DELETE request for a room multiple times.*

    - DELETE requests are idempotent in nature as repeating them leads to the identical system state in all cases. As far as this specific implementation goes, the first DELETE request leads to successful removal of the room. Repeating the request will result in the room being non-existent and hence getting back a response stating that the room is not found. But since nothing will change after the first time the room gets deleted, it satisfies the idempotent requirement. Moreover, business logic will not allow deletion when there are sensors present.

Part 3: Sensor Operations & Linking

1) *We explicitly use the @Consumes (MediaType.APPLICATION_JSON) annotation on the POST method. Explain the technical consequences if a client attempts to send data in a different format, such as text/plain or application/xml. How does JAX-RS handle this mismatch?*

    - The @Consumes(MediaType.APPLICATION_JSON) annotation indicates that the resource method will accept only those requests that have a Content-Type header value set to application/json.
    - Requests that may contain any other kind of data, like text/plain or application/xml, will not be considered by the JAX-RS runtime, and an HTTP 415 Unsupported Media Type status will be returned. In this way, the input data is strictly validated before being processed.

2) *You implemented this filtering using @QueryParam. Contrast this with an alternative design where the type is part of the URL path (e.g., /api/vl/sensors/type/CO2). Why is the query parameter approach generally considered superior for filtering and searching collections?*

    - Filtering of the sensor data would be done through query parameters, such as: GET /api/v1/sensors?type=CO2
    - Alternatively, path parameters could be used for the same purpose: GET /api/v1/sensors/type/CO2
    - The use of query parameters is more fitting for filtering because it is an optional process that may involve different filter criteria being applied at once. It is designed primarily for querying collections rather than pointing to a resource.
    - On the other hand, path parameters are meant to identify a specific resource. Thus, applying the query parameter for filtering purposes is a proper solution.

Part 4: Deep Nesting with Sub – Resources

1) *Discuss the architectural benefits of the Sub-Resource Locator pattern. How does delegating logic to separate classes help manage complexity in large APIs compared to defining every nested path (e.g., sensors/{id}/readings/{rid}) in one massive controller class?*

    - In the Sub-Resource Locator design pattern, the management of sub-resources is delegated to distinct classes. According to this design pattern, any request made to /sensors/{id}/readings will be managed by another class called SensorReadingResource. The advantage of this pattern is that it promotes the concept of modular design, where each class is responsible for one function. It is scalable because any change made to the readings logic does not impact the overall sensor resource.

Part 5: Advanced Error Handling, Exception Mapping & Logging

1) *Why is HTTP 422 often considered more semantically accurate than a standard 404 when the issue is a missing reference inside a valid JSON payload?*
    - HTTP response code 422 is more semantic than 404 for the given situation because although the request in question is fine, its content references an invalid resource. For instance, if one attempts to create a sensor using an invalid room ID, the request is formatted correctly but there's no such room as referenced in it. HTTP response code 404 means that the specified resource was not found, while code 422 means that the request cannot be processed due to incorrect input data.

2) *From a cybersecurity standpoint, explain the risks associated with exposing internal Java stack traces to external API consumers. What specific information could an attacker gather from such a trace?*

    - The problem of exposing stack traces of Java code to consumers of an external API results in huge security threats. The stack trace exposes critical information like the path of the files, classes used, methods used, and internal structure of the program. With such information in their possession, the attacker can easily study the internal structure of the system and exploit its weaknesses. It is possible to prevent exposure of such critical information by creating a global exception handler that sends a generic response.

3) *Why is it advantageous to use JAX-RS filters for cross-cutting concerns like logging, rather than manually inserting Logger.info() statements inside every single resource method?*

- The logging implementation is done by leveraging the JAX-RS filters, which are ContainerRequestFilter and ContainerResponseFilter.
- It allows you to put the logging code in one place so that logging of each API invocation can be done uniformly without code repetition within the resource methods.
- It also helps improve the readability and maintenance of the resource classes as well as avoid any code duplication.